
MEA OCPP 1.6 Compliance Test

Setup & Run Manual

Vector vSECC.single Board vs MEA CSMS Sandbox
Sections 2–11 | Direct connection (no proxy)

Device under test	Vector vSECC.single Board
Device IP	192.168.1.166
OCPP version	1.6 (JSON over WebSocket)
CP identity	rddQC4000001
CSMS	wss://ocpp.measandbox.com:2930
Test PC	192.168.111.185 (+ alias 192.168.1.200/24)
Document	docs/vsecc.test_manual.pdf

Contents

1	System Overview	2
1.1	Architecture Diagram	2
1.2	How it works	2
2	Hardware Requirements	2
3	Network Setup	3
3.1	Physical wiring	3
3.2	Verify connectivity	3
3.3	Make the alias persistent	3
4	Software Prerequisites	3
4.1	Python packages	3
4.2	pdflatex (for PDF reports)	3
4.3	MQTT monitor (optional, for debugging)	3
5	Configuration	4
6	Running the Tests	4
6.1	Section guide	4
6.2	Run all sections without EV (quick validation)	5
6.3	Run all sections with real EV	5
7	Running With a Real EV — Step by Step	5
7.1	Pre-run checklist	5
7.2	Section 3 — Normal Operation	6
7.3	Section 4 — Reset Check	6
7.4	Section 7 — Abnormal Operation	6
8	Building PDF Reports	6
9	Interpreting Results	7
9.1	Result codes	7
9.2	Known MEA sandbox limitations	7
10	Troubleshooting	7
11	Quick Reference	8
12	File Structure	10

1 System Overview

1.1 Architecture Diagram

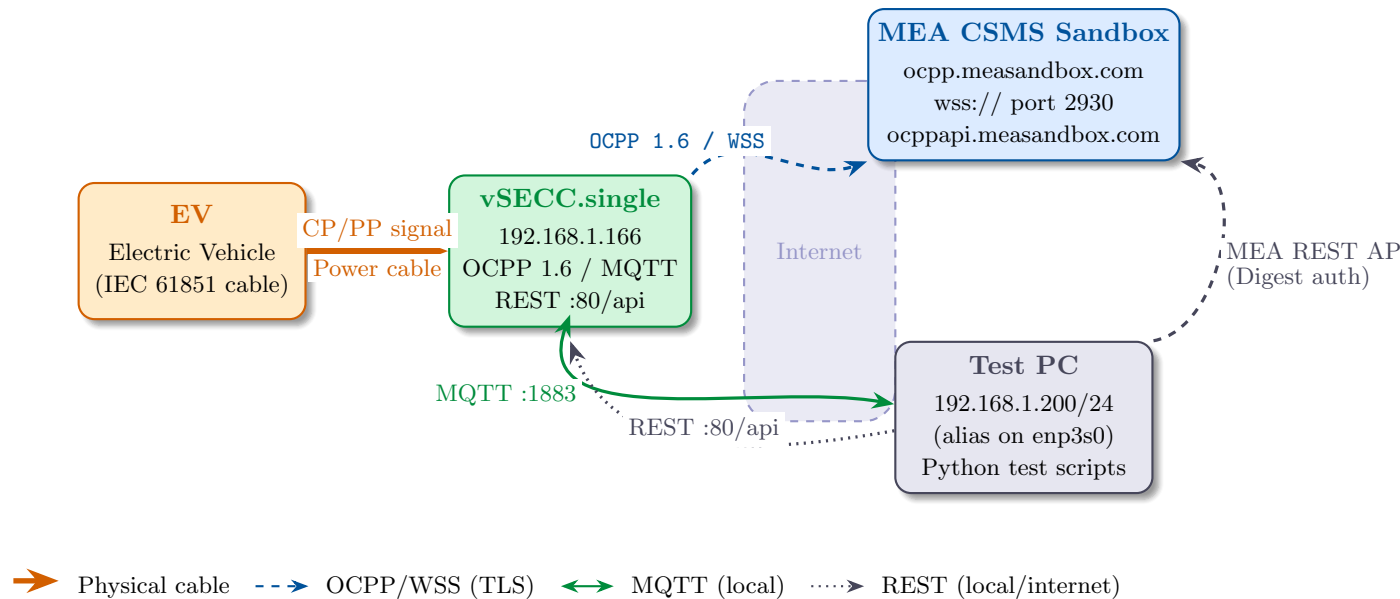


Figure 1: MEA vSECC Test System — Network and Protocol Topology

1.2 How it works

The vSECC.single Board handles CP signalling, SLAC, and ISO 15118 autonomously. It connects directly to the MEA CSMS sandbox over a TLS WebSocket (OCPP 1.6 JSON). The test PC sits on the same local subnet and:

- Observes OCPP events via the **vSECC MQTT broker** on port 1883.
- Reads vSECC state via the **vSECC REST API** on port 80.
- Sends OCPP commands (RemoteStart, Reset, ReserveNow, ...) via the **MEA REST API** over HTTPS.

No OCPP proxy is used. All OCPP traffic runs directly between vSECC and CSMS.

2 Hardware Requirements

Component	Details
Vector vSECC.single	Firmware with OCPP 1.6 JSON; MQTT broker enabled
Test PC	Linux, Python 3.11+, pdf _l at _e x
Network switch	Connects vSECC and PC on the 192.168.1.x subnet
EV (optional)	Required only for EV-plug test items; all API tests run without it

The vSECC must be pre-configured with:

- OCPP endpoint: `wss://ocpp.measandbox.com:2930/EV/Srv/JSON/1.6/rddQC4000001`
- CP identity: `rddQC4000001`
- MQTT broker: built-in at `192.168.1.166:1883`

3 Network Setup

3.1 Physical wiring

Connect `enp3s0` on the Test PC to the same switch as the vSECC ETH port. The PC gets a DHCP address on `192.168.111.0/24`. Because the vSECC is on `192.168.1.x`, add an alias:

```
sudo ip addr add 192.168.1.200/24 dev enp3s0
```

Note

This alias is **not persistent** across reboots or network restarts. Re-run the command each session, or make it persistent (see below).

3.2 Verify connectivity

```
ping -c 3 192.168.1.166 # vSECC should reply
curl -s http://192.168.1.166/api | head # should return JSON (not timeout)
```

3.3 Make the alias persistent

Create `/etc/networkd-dispatcher/routable.d/10-vsecc-alias`:

```
#!/bin/bash
ip addr add 192.168.1.200/24 dev enp3s0 2>/dev/null || true
```

```
sudo chmod +x /etc/networkd-dispatcher/routable.d/10-vsecc-alias
```

4 Software Prerequisites

4.1 Python packages

```
.venv/bin/pip install requests paho-mqtt
```

4.2 pdflatex (for PDF reports)

```
sudo apt install texlive-latex-base texlive-latex-extra \
    texlive-fonts-recommended
pdflatex --version # verify
```

4.3 MQTT monitor (optional, for debugging)

```
sudo apt install mosquitto-clients

# Subscribe to all relevant vSECC topics:
mosquitto_sub -h 192.168.1.166 -p 1883 -u vector -P vector \
    -t "vsecc/connector/#" \
    -t "vsecc/ocpp_connection_status" -v
```

5 Configuration

All parameters are constants at the top of each test file.

Constant	Default	Description
VSECC_BASE	http://192.168.1.166:8060	vSECC REST endpoint
MQTT_HOST	192.168.1.166	vSECC MQTT broker host
CP_ID	rddQC4000001	Charge point identity
RFID_TAG	RFID_TEST	RFID tag used for auth
EV_WAIT_SEC	60	Seconds to wait for EV plug event
BOOT_WAIT_SEC	90	Seconds to wait after Hard Reset
LOCAL_STOP_WAIT_SEC	30	Seconds to wait for RFID tap to stop session
EXPIRY_WAIT_SEC	90	Seconds to wait for reservation expiry (sec. 5)

Without a real EV — set `EV_WAIT_SEC = 0`. EV-dependent items record **WARN** immediately instead of blocking.

```
sed -i 's/^EV_WAIT_SEC = 60/EV_WAIT_SEC = 0/' \
  tests/vsecc/test_mea_sectionN.py
```

6 Running the Tests

All test scripts are standalone Python programs (not pytest). Run from the **project root**:

```
.venv/bin/python3 tests/vsecc/test_mea_sectionN.py
```

Results are saved automatically to `tex/vsecc_sectionN_results.json`.

6.1 Section guide

Sec.	Title	EV needed	Notes
2	Auto Charge	No	Verifies OCPP boot + auto-auth
3	Normal Operation	Yes (3.3+)	Local RFID + RemoteStart flows
4	Reset Check	Yes (4.4+)	Hard & Soft Reset; vSECC reboots
5	Reservation	Yes (5.11+)	Reserve→Cancel, Expiry, Charge
6	Charging Profile	Yes (6.2+)	SetChargingProfile during session
7	Abnormal Operation	Yes (7.2+)	E-stop, door, power loss, local list
8	Dual Connector	N/A	All SKIP — single connector device
9	V2G / BPT	No	ChangeConfiguration + metervalues
10	CSMS Commands	No	All 23 OCPP commands
11	Performance	No	Reconnect time (3× Hard Reset)

6.2 Run all sections without EV (quick validation)

```
for s in 2 3 4 5 6 7 8 9 10 11; do
    sed -i 's/^EV_WAIT_SEC = 60/EV_WAIT_SEC = 0/' \
        tests/vsecc/test_mea_section${s}.py 2>/dev/null
    echo "=== Section $s ==="
    .venv/bin/python3 tests/vsecc/test_mea_section${s}.py
done
```

6.3 Run all sections with real EV

```
# Restore EV wait timeout:
for s in 2 3 4 5 6 7; do
    sed -i 's/^EV_WAIT_SEC = 0/EV_WAIT_SEC = 60/' \
        tests/vsecc/test_mea_section${s}.py 2>/dev/null
done

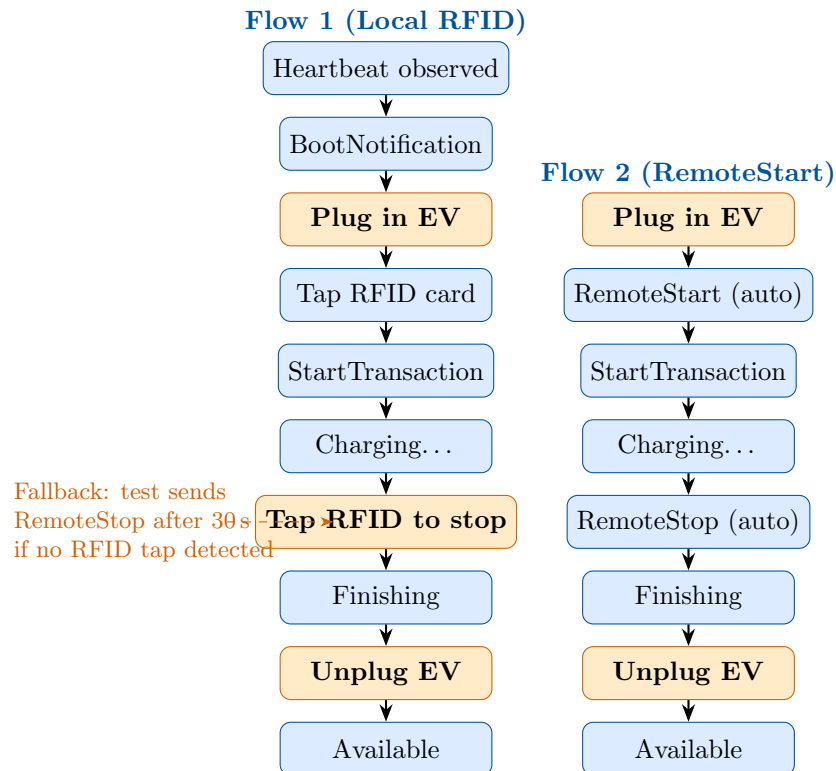
# Run all sections:
for s in 2 3 4 5 6 7 8 9 10 11; do
    echo "=== Section $s ==="
    .venv/bin/python3 tests/vsecc/test_mea_section${s}.py
done
```

7 Running With a Real EV — Step by Step

7.1 Pre-run checklist

- ☐ vSECC powered on; OCPP LED green (connected to MEA CSMS).
- ☐ Network alias active: `ip addr show enp3s0 | grep 192.168.1.200`
- ☐ EV not at 100% SoC (must be willing to accept a session).
- ☐ RFID card `RFID_TEST` registered in MEA CSMS, or use a physical authorised card and update `RFID_TAG`.
- ☐ `EV_WAIT_SEC = 60` in the test file.

7.2 Section 3 — Normal Operation



7.3 Section 4 — Reset Check

After items 4.1 and 4.18 (Hard Reset), the vSECC physically reboots. The test waits up to `BOOT.WAIT_SEC` (default 90s) for the OCPP LED to go green before continuing. No manual action is required; simply wait.

7.4 Section 7 — Abnormal Operation

Some items require physical interaction with the vSECC hardware:

Item	Action	Detail
7.4	Press E-stop	Press the emergency stop button on the vSECC enclosure
7.5	Open door	Open the vSECC enclosure door (if door sensor fitted)
7.6	Power loss	Simulated via Hard Reset (automatic)
7.7	Local list	Automatic via SendLocalList REST call

If a hardware action cannot be performed, the item records **WARN**.

8 Building PDF Reports

After running a test (which saves `tex/vsecc_sectionN_results.json`):

```
# Single section:
.venv/bin/python3 tests/vsecc/generate_section5_report.py
```

```
# All sections at once:
for s in 2 3 4 5 6 7 8 9 10 11; do
    .venv/bin/python3 tests/vsecc/generate_section${s}_report.py
done
```

Output files are written to the `tex/` directory:

File	Section
<code>tex/vsecc_section2_report.pdf</code>	Section 2: Auto Charge
<code>tex/vsecc_section3_report.pdf</code>	Section 3: Normal Operation
<code>:</code>	<code>:</code>
<code>tex/vsecc_section11_report.pdf</code>	Section 11: Performance

9 Interpreting Results

9.1 Result codes

Status	Meaning
PASS	Command accepted / MQTT event observed as expected.
FAIL	Command rejected, or REST endpoint returned an HTTP error.
WARN	Event not observed within the timeout, or a physical step was required but not performed (no EV connected, no RFID tap, etc.).
SKIP	Test not applicable (Section 8: dual-connector tests on a single-connector device).

9.2 Known MEA sandbox limitations

The following REST endpoints return HTTP 404 on the MEA sandbox. These FAILs reflect API coverage, **not** vSECC compliance issues.

OCPP message	MEA REST endpoint	Status
TriggerMessage	<code>/remote/triggerMessage</code>	404
ChangeAvailability	<code>/remote/changeAvailability</code>	404
GetDiagnostics	<code>/remote/getDiagnostics</code>	404
UpdateFirmware	<code>/remote/updateFirmware</code>	404
SendLocalList	<code>/remote/sendLocalList</code>	404
GetLocalListVersion	<code>/cmd/chargepoint/getLocalListVersion</code>	404

10 Troubleshooting

vSECC not reachable (Connection timeout)

```
# Check alias:
ip addr show enp3s0 | grep 192.168.1
# Re-add if missing:
sudo ip addr add 192.168.1.200/24 dev enp3s0
# Verify:
ping 192.168.1.166
```

vSECC not connected to MEA CSMS

Check the OCPP LED on the board. The test prints **ERROR: vSECC not connected to MEA CSMS** and marks all items **FAIL**.

```
# Monitor OCPP connection:
mosquitto_sub -h 192.168.1.166 -u vector -P vector \
  -t "vsecc/ocpp_connection_status"
```

MQTT events not received

Ensure the PC is on the 192.168.1.x subnet (alias active). The vSECC MQTT broker is only accessible on the local network.

```
mosquitto_sub -h 192.168.1.166 -p 1883 -u vector -P vector \
  -t "vsecc/connector/1/status/#" -v
```

Hard Reset — vSECC does not reconnect

After Hard Reset the vSECC reboots in 30–90 s. If it does not reconnect within `BOOT_WAIT_SEC`, check:

- vSECC power is stable.
- OCPP endpoint URL is correctly configured on the vSECC.
- MEA CSMS sandbox is reachable from the vSECC network.

Test hangs / Ctrl+C needed

The test may wait for an MQTT event that will never arrive (e.g. EV plug without a connected EV, or a session event during an idle state). Press **Ctrl+C** to abort. Partial results are **not** saved. Resolve the issue and rerun.

11 Quick Reference

Complete session — copy and paste

```
# 1. Add network alias (required every session):
sudo ip addr add 192.168.1.200/24 dev enp3s0

# 2. Verify vSECC is reachable:
ping -c 2 192.168.1.166

# 3. (Optional) Monitor MQTT in a second terminal:
mosquitto_sub -h 192.168.1.166 -u vector -P vector \
  -t "vsecc/connector/#" -t "vsecc/ocpp_connection_status" -v

# 4. Run a single section:
.venv/bin/python3 tests/vsecc/test_mea_section3.py

# 5. Build its PDF report:
.venv/bin/python3 tests/vsecc/generate_section3_report.py

# 6. Run all sections + build all PDFs in one go:
for s in 2 3 4 5 6 7 8 9 10 11; do
```

```
.venv/bin/python3 tests/vsecc/test_mea_section${s}.py  
.venv/bin/python3 tests/vsecc/generate_section${s}_report.py  
done
```

12 File Structure

```
MEA-V2G/
|-- tests/vsecc/
| |-- test_mea_section2.py Section 2: Auto Charge
| |-- test_mea_section3.py Section 3: Normal Operation
| |-- test_mea_section4.py Section 4: Reset Check
| |-- test_mea_section5.py Section 5: Reservation
| |-- test_mea_section6.py Section 6: Charging Profile
| |-- test_mea_section7.py Section 7: Abnormal Operation
| |-- test_mea_section8.py Section 8: Dual Connector (all SKIP)
| |-- test_mea_section9.py Section 9: V2G / BPT
| |-- test_mea_section10.py Section 10: CSMS Commands
| |-- test_mea_section11.py Section 11: Performance
| |-- generate_section2_report.py
| |-- generate_section3_report.py
| |-- ...
| '-- generate_section11_report.py
|-- tex/
| |-- vsecc_section2_results.json raw results (auto-generated)
| |-- vsecc_section2_report.pdf PDF report (auto-generated)
| '-- ...
'-- docs/
    |-- vsecc_test_manual.md Markdown version of this manual
    '-- vsecc_test_manual.pdf This document
```